

Indice

1. Introducción a la memoria.....	3
1.2 Descripción.....	3
1.3 Objetivos generales.....	3
1.4 Beneficios.....	3
1.5 Motivaciones personales.....	3
1.6 Estructura de la memoria.....	4
2. Estudio de viabilidad.....	4
2.1 Introducción.....	4
2.1.1 Tipología y palabras clave.....	4
2.1.2 Descripción.....	4
2.1.3 Objetivos del proyecto.....	5
2.1.4 Clasificación de los objetivos.....	5
2.1.5 Definiciones, acrónimos y abreviaciones.....	5
2.1.6 Partes interesadas.....	5
3. METODOLOGÍA DE DESARROLLO DEL PROYECTO.....	6
3.1. Naturaleza práctica del proyecto.....	6
3.2. Flexibilidad en la implementación de servicios.....	6
3.3. Reducción de riesgos.....	6
3.4. Facilita pruebas reales desde el inicio.....	6
4. Estudio de alternativas.....	7
4.1. Servidores tradicionales o dedicados.....	7
4.2. Equipos sobremesa reciclados.....	7
4.3. Microservidores comerciales.....	7
4.4. Raspberry Pi como servidor.....	7
5. Objetivos parciales.....	7
5.1. Preparación del hardware.....	8
5.2. Instalación de la plataforma de contenedorización.....	8
5.3. Configuración de Docker Compose.....	8
5.4. Implementación de servicios en contenedores.....	8
5.5. Configuración de redes y volúmenes.....	8
5.6. Gestión y monitorización.....	8
5.7. Pruebas de funcionamiento.....	8
5.8. Documentación.....	8
6. Permite una documentación mejor estructurada.....	8
7. Adecuada para proyectos educativos.....	9
8. Planificación Temporal del Proyecto.....	9
8.1. Decisión de la metodología y justificación.....	9
8.2. Elección de herramientas y justificación.....	9
8.3. Planteamiento de fases del proyecto.....	9
Fase 1: Análisis y planificación inicial.....	9
Fase 2: Emulación de la Raspberry Pi y documentación.....	10
Fase 3: Pruebas finales y validación en entorno emulado.....	10
Fase 4: Implementación en Raspberry Pi real.....	10
Fase 5: Revisión final y cierre del proyecto.....	10
9. Control de Versiones.....	10
9.1 Herramienta utilizada.....	10
9.2 Beneficios del control de versiones.....	11
10. Mecanismos de Seguridad y Recuperación.....	11
10.1 Seguridad del sistema operativo.....	11
10.2 Seguridad en Docker.....	11

10.3 Seguridad de red.....	11
11. Mecanismos de recuperación.....	11
11.1 Copias de seguridad.....	11
11.2 Recuperación ante fallos.....	12

1. Introducción a la memoria

En esta memoria se presenta el desarrollo de un proyecto cuyo objetivo es convertir una Raspberry Pi en un servidor ligero utilizando Docker. Debido a su bajo coste, tamaño reducido y mínimo consumo energético, la Raspberry Pi se plantea como una excelente alternativa a los servidores tradicionales, que suelen requerir hardware costoso y un mayor gasto de mantenimiento. A lo largo del documento se describe el análisis, diseño, implementación y organización del sistema, así como los servicios que podrán ejecutarse dentro de contenedores Docker.

1.2 Descripción

El proyecto consiste en instalar un sistema operativo minimalista en una Raspberry Pi y configurarla como servidor mediante Docker. Docker permitirá ejecutar servicios aislados, fáciles de gestionar y con un mantenimiento sencillo, aprovechando al máximo los recursos limitados del dispositivo. Además, se utilizará Docker Compose para organizar los contenedores, redes, volúmenes y dependencias. De forma opcional, se podrá añadir un panel de administración como Portainer para facilitar la supervisión del servidor.

1.3 Objetivos generales

- Convertir una Raspberry Pi en un servidor funcional y estable.
- Instalar y configurar Docker como plataforma de contenedorización.
- Gestionar servicios mediante Docker Compose para facilitar su despliegue y organización.
- Crear una arquitectura modular que permita añadir, modificar o eliminar servicios fácilmente.
- Garantizar un funcionamiento eficiente, continuo y con bajo consumo energético.
- Documentar todo el proceso técnico del proyecto.

1.4 Beneficios

- **Ahorro económico:** los costes de hardware y mantenimiento son muy inferiores a los de un servidor tradicional.
- **Bajo consumo energético:** la Raspberry Pi consume muy poca electricidad en funcionamiento continuo.
- **Modularidad y flexibilidad:** cada servicio se ejecuta en su propio contenedor, evitando conflictos y facilitando actualizaciones.
- **Facilidad de gestión:** con Docker Compose se controla todo el sistema desde una configuración centralizada.
- **Posibilidad de ampliación:** se pueden añadir nuevos servicios sin modificar el sistema principal.

1.5 Motivaciones personales

La motivación principal del proyecto es demostrar que es posible crear un servidor completo utilizando hardware de bajo coste y tecnologías modernas como Docker. Además, permite adquirir experiencia práctica en administración de sistemas, redes, seguridad, automatización y contenedorización, todas ellas competencias relevantes en el ámbito profesional. También supone la oportunidad de aprender a configurar un servidor real de forma ligera, eficiente y estable.

1.6 Estructura de la memoria

La memoria se organiza en diferentes apartados que permiten entender el desarrollo del proyecto:

1. Introducción y descripción del objetivo del proyecto.
2. Análisis de las soluciones existentes y justificación del uso de Raspberry Pi y Docker.
3. Análisis y diseño del sistema, especificando hardware, contenedores y capas de la arquitectura.
4. Desarrollo del proyecto, detallando instalación, configuración y servicios.
5. Estimación de costes y recursos necesarios.
6. Contenidos transversales y competencias adquiridas.

2. Estudio de viabilidad

El estudio de viabilidad evalúa si el proyecto de convertir una Raspberry Pi en un servidor ligero mediante Docker puede llevarse a cabo con los recursos disponibles, valorando factores técnicos, económicos y funcionales. También analiza las motivaciones del proyecto, sus objetivos y las partes implicadas.

2.1 Introducción

Este apartado presenta el contexto general del proyecto y establece los elementos necesarios para su análisis. Aquí se identifican el tipo de proyecto, su descripción básica, los objetivos, las definiciones relevantes y los actores que intervienen directa o indirectamente en su realización. El propósito es determinar si el sistema es viable en términos técnicos, económicos y operativos.

2.1.1 Tipología y palabras clave

- **Tipología del proyecto:**
Proyecto técnico de instalación, configuración y administración de un servidor ligero basado en contenedores.
- **Palabras clave:**
Raspberry Pi, Docker, Docker Compose, servidor ligero, contenedores, Portainer, administración de sistemas, bajo consumo, hardware económico.

2.1.2 Descripción

El proyecto consiste en configurar una Raspberry Pi con un sistema operativo ligero (Raspberry Pi OS Lite) e instalar Docker para ejecutar distintos servicios dentro de contenedores aislados. La utilización de Docker permite gestionar de forma modular cada servicio, evitándose conflictos y facilitando su actualización. Docker Compose se usará para coordinar los contenedores, sus redes y sus volúmenes. Además, opcionalmente se puede integrar un panel como Portainer para supervisar el sistema.

El objetivo es disponer de un servidor funcional, económico y de bajo consumo energético.

2.1.3 Objetivos del proyecto

Los objetivos, basados en el documento original, son:

- Crear un servidor ligero y funcional utilizando una Raspberry Pi.
- Instalar y configurar Docker como herramienta principal para ejecutar servicios en contenedores.
- Gestionar los contenedores mediante Docker Compose para facilitar su organización.
- Desplegar servicios independientes en contenedores (servidor web, base de datos, DNS, nube privada, etc.).
- Garantizar estabilidad, modularidad y bajo consumo energético.
- Facilitar el mantenimiento y la futura ampliación del servidor.

2.1.4 Clasificación de los objetivos

- **Objetivos técnicos:**
 - Implementar Docker y Docker Compose.
 - Ejecutar servicios en contenedores aislados.
 - Configurar la Raspberry Pi como servidor estable.
- **Objetivos económicos:**
 - Reducir costes usando hardware económico.
 - Minimizar el consumo eléctrico.
- **Objetivos funcionales:**
 - Permitir la ejecución continua 24/7.
 - Ofrecer una arquitectura modular y escalable.
- **Objetivos formativos:**
 - Aprender administración de sistemas y redes.
 - Adquirir experiencia con contenedores y automatización.
 - Mejorar competencias técnicas y digitales.

2.1.5 Definiciones, acrónimos y abreviaciones

- **Raspberry Pi:** miniordenador de bajo consumo utilizado como servidor.
- **Docker:** plataforma de contenedorización que permite ejecutar aplicaciones de forma aislada.
- **Docker Compose:** herramienta para definir y ejecutar múltiples contenedores.
- **OS (Operating System):** sistema operativo.
- **Portainer:** panel gráfico de administración de contenedores Docker.
- **SMR:** Sistemas Microinformáticos y Redes.

2.1.6 Partes interesadas

- Usuarios que quieran hacer un servidor de este tipo
- Empresas
- Centros educativos

3. METODOLOGÍA DE DESARROLLO DEL PROYECTO

Para el desarrollo del proyecto se empleará una **metodología incremental y basada en prototipos**, que resulta especialmente adecuada para proyectos tecnológicos y de configuración de sistemas como la creación de un servidor Docker en Raspberry Pi. Esta metodología permite avanzar de forma progresiva, validando cada etapa mientras se construye el sistema final. La elección de esta metodología se justifica por los siguientes motivos:

3.1. Naturaleza práctica del proyecto

El objetivo principal del proyecto es instalar, configurar y desplegar servicios dentro de contenedores Docker sobre una Raspberry Pi. Se trata de un trabajo altamente técnico, donde es necesario **comprobar el funcionamiento real del sistema en cada fase**.

El enfoque incremental facilita esta verificación continua y permite detectar errores tempranamente.

3.2. Flexibilidad en la implementación de servicios

Al utilizar Docker, el proyecto se compone de **bloques independientes (contenedores)**.

La metodología incremental encaja perfectamente, ya que cada contenedor se puede:

- Diseñar
- Implementar
- Probar
- Integrar

de forma individual antes de añadir nuevos servicios al sistema.

3.3. Reducción de riesgos

Trabajar con hardware de bajo consumo como la Raspberry Pi implica limitaciones de recursos.

Mediante incrementos controlados se puede:

- Evaluar el consumo de CPU y RAM por cada servicio
- Ajustar configuraciones
- Optimizar el rendimiento
- Evitar saturaciones del sistema

Esta validación continua reduce riesgos frente a metodologías más rígidas.

3.4. Facilita pruebas reales desde el inicio

Desde las primeras fases ya se obtiene un **prototipo funcional**: primero la Raspberry Pi con el sistema operativo, luego Docker, después el primer servicio contenedor, etc.

Esto permite comprobar rápidamente si el proyecto avanza en la dirección correcta sin esperar al desarrollo completo.

4. Estudio de alternativas

Antes de elegir la Raspberry Pi como plataforma para el desarrollo del servidor, es necesario analizar las alternativas disponibles. En el documento se menciona que existen muchos tipos de servidores Docker, pero la mayoría requieren **hardware costoso, mayor consumo energético y gastos de mantenimiento elevados**. Estas características los hacen poco adecuados para un entorno doméstico, educativo o de bajo presupuesto.

Entre las alternativas consideradas estarían:

4.1. Servidores tradicionales o dedicados

- Requieren una inversión inicial elevada.
- Consumen más energía, lo que incrementa su coste de uso continuo.
- Pueden tener un mantenimiento más complejo y costoso.

Motivo para descartarlos: no cumplen con los objetivos de bajo coste y eficiencia energética.

PropuestaProyecto

4.2. Equipos sobremesa reciclados

- Aunque reutilizar un PC antiguo es una opción, normalmente consume mucha más electricidad que una Raspberry Pi.
- El hardware suele ser voluminoso y posiblemente ruidoso.

Motivo para descartarlos: superan los límites de consumo y tamaño buscados.

4.3. Microservidores comerciales

- Existen modelos compactos, pero nuevamente son más caros que una Raspberry Pi y algunos necesitan mayor potencia eléctrica.

Motivo para descartarlos: rompen la idea de un servidor económico y accesible.

4.4. Raspberry Pi como servidor

- Hardware barato, accesible y fácil de conseguir.
- Consumo energético extremadamente bajo.
- Ideal para funcionamiento continuo.
- Suficiente potencia para ejecutar contenedores Docker.

Motivo para seleccionarla: combina precio reducido, eficiencia energética y capacidad suficiente para los servicios del proyecto.

5. Objetivos parciales

Estos objetivos derivan directamente de los bloques del proyecto descritos en tu documento (capas, análisis del sistema, servicios, etc.), Se trata de metas intermedias necesarias para completar el proyecto general.

5.1. Preparación del hardware

- Configurar la Raspberry Pi como servidor base.
- Instalar el sistema operativo Raspberry Pi OS Lite.

5.2. Instalación de la plataforma de contenedorización

- Instalar Docker correctamente en la Raspberry Pi.
- Verificar que Docker funciona y permite ejecutar contenedores simples.

5.3. Configuración de Docker Compose

- Instalar Docker Compose.
- Crear el archivo principal de configuración para gestionar los servicios.

5.4. Implementación de servicios en contenedores

- Desplegar los primeros contenedores básicos (por ejemplo: servidor web, base de datos, DNS o servicios necesarios).
- Garantizar que cada contenedor funciona de manera aislada sin conflictos.

5.5. Configuración de redes y volúmenes

- Establecer redes internas entre contenedores.
- Definir volúmenes persistentes para conservar datos.

5.6. Gestión y monitorización

- (Opcional) Instalar Portainer u otra herramienta gráfica.
- Monitorizar logs, uso de recursos y estado de los contenedores.

5.7. Pruebas de funcionamiento

- Comprobar que los servicios arrancan automáticamente.
- Verificar estabilidad, rendimiento y consumo energético.

5.8. Documentación

- Registrar cada proceso realizado.
- Redactar la memoria final del proyecto con resultados y conclusiones.

6. Permite una documentación mejor estructurada

Cada incremento añade funciones nuevas al proyecto; por tanto, la metodología facilita documentar:

- Cambios
- Configuraciones aplicadas
- Servicios añadidos
- Mejoras realizadas

El resultado es una memoria final más clara y ordenada.

7. Adecuada para proyectos educativos

En un entorno académico, esta metodología fomenta:

- la planificación por fases,
- el aprendizaje progresivo,
- la evaluación continua del trabajo,
- la reflexión sobre mejoras.

8. Planificación Temporal del Proyecto

8.1. Decisión de la metodología y justificación

Metodología que he elegido es una metodología incremental, la cual trata de como “partir” el proyecto en partes, para ir incrementandola poco a poco, antes de hacerlo en la raspberry pi real, y con este enfoque permite detectar y corregir errores en una fase temprana, reducir riesgos técnicos y asegurar que la implementación final en la Raspberry Pi se realice sobre una base previamente probada y documentada.

8.2. Elección de herramientas y justificación

- **Sistema operativo anfitrión:** Windows 10
- **Emulación de hardware:** QEMU
- **Sistema operativo emulado:** Raspberry Pi OS Lite (32-bit)
- **Plataforma de contenedores:** Docker
- **Gestión de servicios:** Docker Compose

Estas herramientas permiten reproducir de forma fiel el entorno de una Raspberry Pi real y facilitan la migración directa del sistema una vez validado.

8.3. Planteamiento de fases del proyecto

Fase 1: Análisis y planificación inicial

Duración estimada: 2 semanas

- Definición del alcance y objetivos del proyecto.
- Análisis de requisitos técnicos.
- Diseño inicial de la arquitectura del sistema.
- Planificación de la fase de emulación.
- Inicio de la documentación (introducción y objetivos).

Fase 2: Emulación de la Raspberry Pi y documentación

Duración estimada: 2 meses

- Instalación y configuración de QEMU en Windows 10.
- Preparación y arranque de Raspberry Pi OS en entorno emulado.
- Configuración de red y acceso al sistema.
- Instalación de Docker y Docker Compose.
- Pruebas de contenedores compatibles con arquitectura ARM.
- **Documentación completa del proceso de emulación y configuración.**
- Registro de incidencias y soluciones.

Fase 3: Pruebas finales y validación en entorno emulado

Duración estimada: 1 mes

- Pruebas funcionales de los servicios Docker.
- Verificación de conectividad, puertos y persistencia de datos.
- Ajustes de configuración.
- **Documentación de resultados y validación final del entorno emulado.**

Fase 4: Implementación en Raspberry Pi real

Duración estimada: 3 semanas

- Instalación de Raspberry Pi OS en la Raspberry Pi física.
- Instalación de Docker y Docker Compose.
- Despliegue de los servicios previamente validados.
- Verificación de funcionamiento en entorno real.
- Comparativa entre entorno emulado y entorno real.
- Documentación del proceso de migración.

Fase 5: Revisión final y cierre del proyecto

Duración estimada: 2–3 semanas

- Revisión y unificación de toda la documentación.
- Correcciones de formato y coherencia.
- Elaboración de conclusiones.
- Propuestas de mejora futura.

9. Control de Versiones

Los sistemas de control de versiones son software que ayudan a realizar un seguimiento de los cambios realizados en el código a lo largo del tiempo. Como desarrollador edita código, el sistema de control de versiones toma una instantánea de los archivos.

9.1 Herramienta utilizada

Para el control de versiones del proyecto he utilizado **GitHub**, con el objetivo de gestionar de forma eficiente los cambios realizados tanto en la configuración del sistema como en la documentación,

9.2 Beneficios del control de versiones

El control de versiones tiene muchos beneficios, ya que nos permite llevar un recuento de todo lo que llevamos hecho, facilita la recuperación de documentos ya que nos permite acceder al GitHub desde cualquier sitio y dispositivo y nos permite replicar entornos de todo tipo

10. Mecanismos de Seguridad y Recuperación

10.1 Seguridad del sistema operativo

En el sistema Raspberry Pi OS (emulado y real) se aplican las siguientes medidas básicas de seguridad:

- Actualización periódica del sistema operativo
- Uso de cuentas de usuario sin privilegios para tareas habituales.
- Cambio de la contraseña por defecto del usuario.

10.2 Seguridad en Docker

Se han adoptado las siguientes medidas de seguridad en el entorno Docker:

- Uso exclusivo de **imágenes oficiales y confiables**, compatibles con arquitectura ARM.
- Ejecución de contenedores con el mínimo de privilegios necesarios.
- Uso de archivos `docker-compose.yml` para controlar puertos, volúmenes y redes.
- Separación de servicios en contenedores independientes.

10.3 Seguridad de red

- Exposición únicamente de los puertos necesarios para el funcionamiento del servicio.
- Uso de red local durante la fase de pruebas.
- Evitación de la exposición directa del sistema a Internet durante la fase de emulación.

11. Mecanismos de recuperación

11.1 Copias de seguridad

Se implementan copias de seguridad de los siguientes elementos:

- Archivos de configuración de Docker.
- Volúmenes de datos persistentes.
- Documentación del proyecto.

Estas copias se almacenan en un disco externo

11.2 Recuperación ante fallos

En caso de fallo del sistema o error de configuración, se establecen los siguientes mecanismos de recuperación:

- Restauración de archivos de configuración desde el repositorio Git.
- Recreación de contenedores Docker mediante Docker Compose.
- Reinstalación del sistema operativo en la Raspberry Pi y despliegue automático del entorno previamente documentado